融合 MoE 算子的 TileLang 实现与优化

赛题一 XH-202608 —— 基于国产软件栈大模型推理前沿算子优化

算子入口：03_源码与复现/kernel/submission.py::run_kernel

评委先看：一句话结论

本方案把 routed-MoE 的两段 GEMM、SwiGLU 和路由权重乘法融合在一个 TileLang 算子中， 围绕 MetaX C500 的 32 KiB LDS / block 与 256 registers / thread 两条硬约束， 推导出 128×128×64 / 256 threads 的主 tile，再用 shared-memory swizzle、权重寄存器暂存和薄块分流降低访存开销。 在不量化、不调用外部设备库、不缓存计算结果的前提下，三个评测配置分别达到 2.68 / 4.72 / 8.90 ms， 线上得分 81.7 / 80.7 / 80.7；本地正确性、padding 未写和提交沙箱检查全部通过。

五个核心问题	方法	证据位置
做了什么	两阶段融合 MoE：gate/up → SwiGLU → down → route	§2、图 2-1
为什么快	128×128×64 tile 保持双 block 驻留；swizzle 消除 MMA bank conflict；寄存器暂存隐藏 up 权重读取	§3.1 -§3.5
是否正确	27 个布局全部通过；最差绝对误差 2.6e-3；padding 行保持未写	§4、02_日志与结果/03_func_test.log
是否合规	只有 TileLang T.gemm 承担矩阵乘；提交文件 import 白名单检查为 OK	§7、01_check_submission.log
如何复现	执行 ./run.sh quick；完整流程执行 ./run.sh	§8、根目录 README.md

后文先讲数据布局和整体结构，再解释每项优化为什么有效，最后给出测量方法、失败尝试和合规自查。

1. 任务与评测契约

1.1 计算内容

对按 expert 分组排布的 token 矩阵，逐 expert 完成 DeepSeek 式 routed-MoE 的前向：

```
out[i] = routed_w[i] · ( silu(x[i] @ gate_w[e]^T) ⊙ (x[i] @ up_w[e]^T) ) @ down_w[e]^T ,  e = expert(i)
```

即两段矩阵乘（d_hidden → d_expert 的 gate/up，d_expert → d_hidden 的 down）夹一个 SwiGLU， 最后乘路由权重写回。全程 fp16 输入输出、fp32 累加。

1.2 数据布局（这是设计的出发点）

评测系统传入的 stacked_expert_tokens 已按 expert 分组，且每个 expert 的行数向上取整到 128 的 倍数（group_padded_offsets 给出 padding 后的前缀和，group_offsets 给出有效行的前缀和）， group_idx_for_bx[bx] 说明第 bx 个 128 行 block 属于哪个 expert。因此：

- block 与 expert 是对齐的：一个 128 行的 tile 永远只属于一个 expert，权重索引在 block 内是常量；
- 存在 padding 行，它们既不能被读进结果、也不能被写（评测的 allclose 覆盖整个输出张量，往 padding 行写 0 也算错，因为参考实现根本没写过那块内存）；
- 路由分布是倾斜的：三个评测配置的 block 数都恰好是 expert 数的 1.5 倍，即约一半 expert 占一个 block、另一半占两个 block，而第二个 block 往往只有 16 -64 行有效。

1.3 三个评测配置

	experts	d_hidden	d_expert	valid tokens	128 行 block 数
Config1	16	2048	8192	2272	24
Config2	32	7168	2048	4544	48
Config3	64	7168	2048	9088	96

Config1 的形状是"反"的（hidden 小、intermediate 大），它 stage 1 的 n 方向有 64 个 tile 而 stage 2 只有 8 个，因此调度上与另外两个配置并不同构 —— 方案中所有按 shape 分支的地方都源于此。

1.4 约束

只允许使用 TileLang 的高层语法完成计算：不得绕过 TileLang 执行设备 kernel、不得用 PyTorch 完成 核心计算、不得使用 H2D/D2H 异步拷贝规避计时、不得跨调用缓存或复用结果、不得针对已知 testcase 写死、不得 T.import_source / T.call_extern / 外部算子库。提交文件还要通过沙箱的静态检查 （模块顶层只允许字面量赋值，只能 import torch 与 tilelang）。第 7 节是逐条自查。

---

2. 总体设计

2.1 两阶段 + 按行数分流的 scope 划分

整个算子是一个 @tilelang.jit 的 T.prim_func，内部包含 2 -4 个 T.Kernel scope（每个 scope 对应一次 launch），按 shape 决定实际生成几个：

```
                ┌──────────────────────── stage 1 ────────────────────────┐
stacked_tokens ─┤ main scope   128 行 tile  | 有效行 > tail_rows 的 block |
gate_w, up_w   ─┤ tail scope    64 行 tile  | 有效行 ≤tail_rows 的 block  ├→up_logits (workspace)
                └─────────────────────────────────────────────────────────┘

                ┌──────────────────────── stage 2 ──────────────────────────┐
up_logits      ─┤ main scope   128 行 tile ×2 个 n-tile 配对                │
down_w         ─┤ tail scope    64 行 tile ×2 个 n-tile 配对                ├→out（只写有效行）
routed_w       ─┘                                                          │
                └──────────────────────────────────────────────────────────┘
```

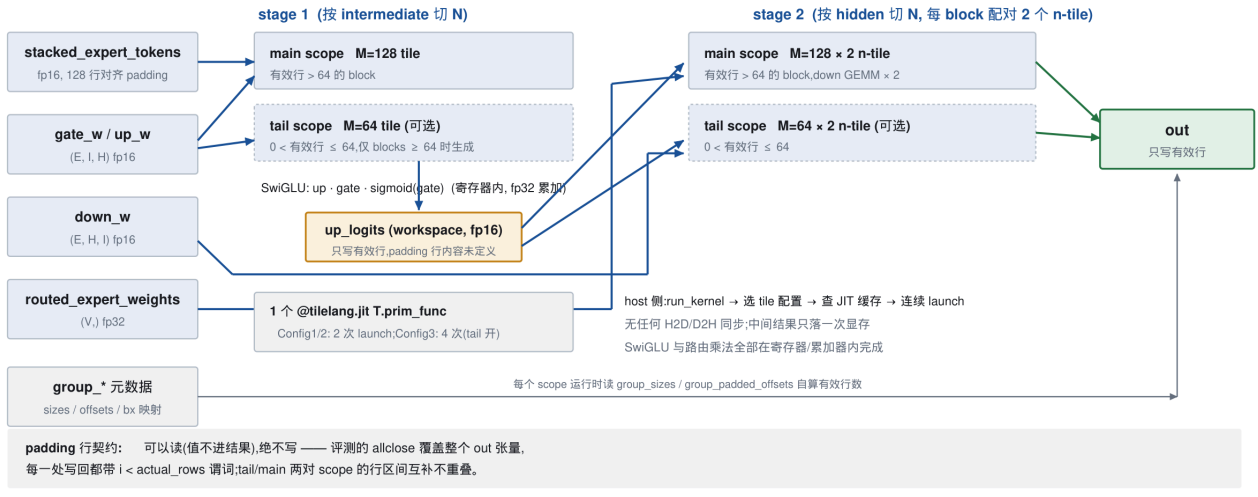


图 2-1 数据流与 scope 划分：两阶段 GEMM 共用一个 fp16 workspace；tail scope 只在 Config3 生成；所有写回都带"仅有效行"谓词

- stage 1：gate 与 up 两次 GEMM 共用同一块 A tile，算完后就地做 SwiGLU，写入 up_logits 这块可复用 workspace；
- stage 2：读 up_logits 做 down GEMM，乘路由权重后直接从累加器写回 out；

- tail scope：一对额外的 scope，用 64 行 tile 处理"有效行 ≤ 64"的薄 block（§3.6），只在 blocks ≥ 64 时启用（Config3），否则不生成，避免多两次 launch。

融合的收益：up_logits 只落一次显存、SwiGLU 与路由权重乘法都在寄存器里完成、两段 GEMM 之间没有 任何 host 同步，一次 run_kernel 只有 2 次（Config1/2）或 4 次（Config3）kernel launch， host 侧总开销 19 μs（02_日志与结果/00_env.log 环境下实测）。

2.2 为什么不是"每 expert 一次 GEMM"

expert 只有 64 ~220 行，单个 GEMM 填不满机器：逐 expert 调用 torch.matmul 实测比本方案慢 1.6 ~2.0 倍（Config1 4.62 / Config2 9.07 / Config3 18.41 ms）。而批量 bmm 需要把每个 expert 的 行数统一到 256，在倾斜路由下要多算 33% 的行，且权重 gather 的代价远超收益。分组 GEMM + 运行时 读取 group_sizes 是唯一能同时避免"填不满"和"多算"的形式。

2.3 技术路线与实现步骤

路线一句话：先固化契约与尺子，再量出硬约束、按约束推导结构，最后做纯语法级优化并用等价性/合规检查收口。分八步，每步都有可核查的产物：

- 固化评测契约（测试先行）：写 03_源码与复现/test/local_bench.py，精确复现评测系统的调用契约 —— padding 后的行布局、fp32 路由权重、n_experts+1 长度的偏移数组、输出区哨兵断言（padding 行不可写）。27 个测试布局（含三个评测 shape 的精确复现）是后面每一步 A/B 的尺子。
- 正确性基线：fp32 参考实现先行，torch 逐 expert 慢版跑通全部功能测试 —— 先有"对"，再追"快"。
- JIT 骨架：单 T.prim_func、两 stage、128 行 block 与 expert 的对齐映射、运行时读 group_sizes / group_padded_offsets 自算有效行；验证 launch 次数与 padding 契约。
- 量出两条硬约束：占用率探针（一块只占地方的 shared buffer）量出每 CU 64 KiB LDS、32 KiB 红线；mcTracer 量出寄存器水位。据此推导出 128×128×64 / 256 线程（§3.1），不是扫参。
- 结构优化迭代：MMA shared tile 的 swizzle 布局（§3.3）→ 权重 tile 寄存器暂存 + 展开因子重配（§3.4/§3.5）→ 运行时薄块分流（§3.6）→ n-tile 配对与 epilogue 融合（§3.7）。每步都做同进程交错 A/B；凡涉及 lowering 层的变体用独立进程 + 独立 TileLang cache 目录（这类改动不进 cache key，共享目录会把旧 kernel 静默发回来）。
- 合规清理（v40）：删除跨调用记忆与 testcase 默认值，换成纯 shape 规则；顺带消灭计时区间内最后一次 D2H 同步（§3.9）。
- 等价性与沙箱自检：check_equiv.py（moe.py 与 submission.py 生成设备码 md5 相同）与 check_submission.py（AST / import 白名单）双双通过。
- 定版复测：最后一轮沿 25 个方向、35 个变体复测全部中性或负收益 → v42 定版；bash run.sh 一键重生成全部日志。

---

3. 关键设计与创新点

3.1 tile 与线程配置：128×128×64 / 256 线程是被约束推导出来的唯一最优点

这是整个方案的地基，也是最值得说明的一点：它不是扫参扫出来的，而是由两条硬件约束推出来的。

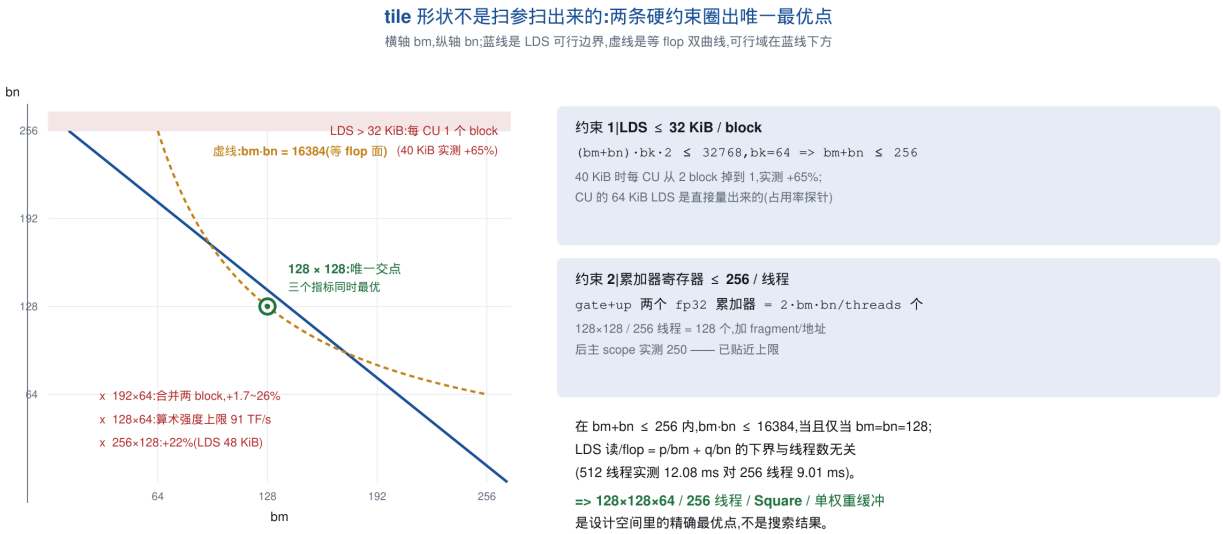


图 3-1 tile 设计空间:LDS 可行边界(蓝线)与等 flop 双曲线(虚线)恰好相切于 128×128;右侧是两条约束的实测依据

记 tile 为 bm×bn×bk，W = threads/64 个 wave 按 p×q = W 切分（每个 wave 负责 bm/p × bn/q）。 三个可量化的指标：

指标	表达式
MMA operand 的 LDS 读 / flop	p/bm + q/bn
灌入 shared memory 的字节 / flop	(bm+2bn) / (2·bm·bn)（A 被 gate/up 共用）
每个 __syncthreads() 摊到的 flop	bm·bn·bk（单权重缓冲时每 k-step 4 个 barrier）

两条硬约束（都在本机实测确认）：

- 每 block 的 LDS ≤ 32 KiB。用一块"只占地方"的 shared buffer 做占用率探针：每 block 32 KiB 时 stage 1 为 3.683 ms，40 KiB 时 6.092 ms（+65%），之后到 64 KiB 完全平坦 —— 也就是超过 32 KiB 就从每 CU 两个 block 掉到一个。于是 (bm+bn)·bk·2 ≤ 32768，bk=64 时 bm+bn ≤ 256。
- 累加器寄存器：gate 与 up 两个 fp32 累加器占 2·bm·bn/threads 个寄存器，mcTracer 实测 stage 1 主 scope 共 250 个/线程，而每 CU 两个 256 线程 block 的上限正好是 256 个。

在 bm+bn ≤ 256 下 bm·bn 的最大值就是 16384（bm=bn=128），三个指标同时取到最优： 读 0.03125 B/flop、写 0.01172 B/flop、每 barrier 1.05 MFLOP。线程数也没有选择：p·q = threads/64 与寄存器允许的 bm·bn 同比例变化，读/flop 的下界 $2\sqrt{(pq/(bm\cdot bn))}$ 与线程数无关（实测 512 线程 12.08 ms 对 256 线程的 9.01 ms，正是因为 wave tile 退化成 32×64）。bk 只影响 DRAM 行段长度 （越大越好）与 LDS 容量（越小越好），64 恰好把 32 KiB 填满，且 128 字节的行段实测比 64 字节 （bk=32）快 24%。

=> 结论：128×128×64 / 256 线程 / Square policy / 单权重缓冲 是这个设计空间里的精确最优点。 第 6 节给出所有偏离它的尝试及实测代价。

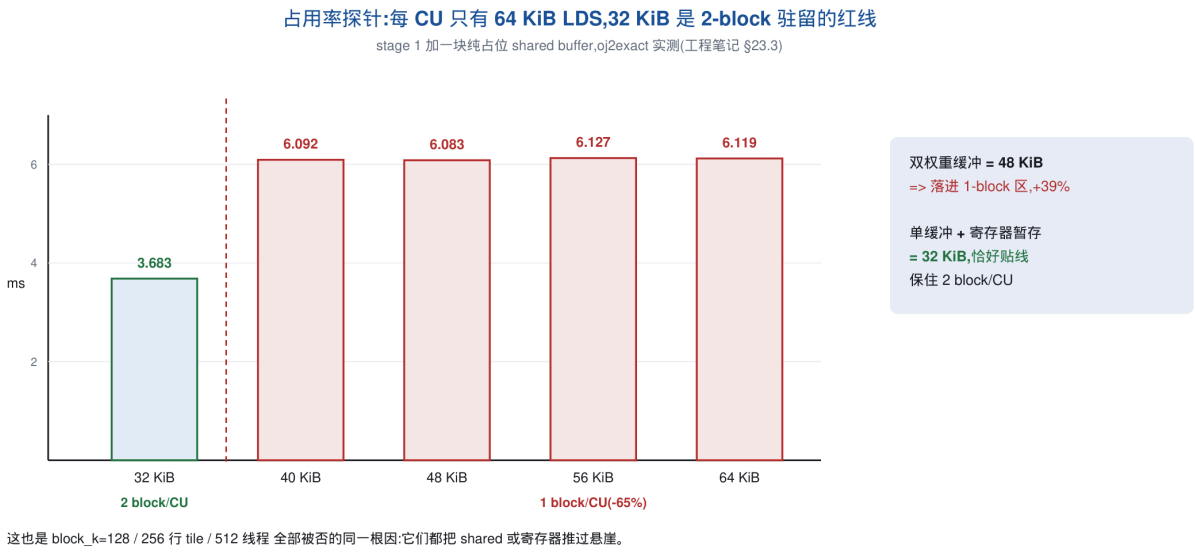


图 3-2 约束 1 的实测:占用率探针量出每 CU 64 KiB LDS,32 KiB 是 2-block 驻留的红线

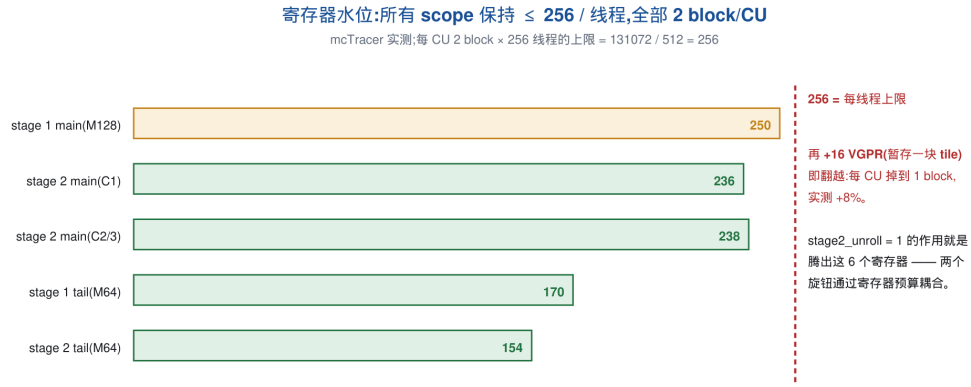


图 3-3 约束 2 的实测:五个 scope 的寄存器水位全部 ≤ 256,主 scope 只剩 6 个余量

3.2 A 操作数共用 + 单权重缓冲：把 32 KiB 花在最值钱的地方

一个 k-step 的结构是：

```
copy A[k]      → input_shared        (128×64 fp16, 16 KiB)
copy gate_w[k] → weight_shared       (128×64 fp16, 16 KiB)
copy up_w[k]   → w_stage（寄存器）     ← §3.4
[barrier]
gemm(input_shared, weight_shared, gate_acc)
[barrier]
copy w_stage   → weight_shared       （复用同一块 shared）
[barrier]
gemm(input_shared, weight_shared, up_acc)
[barrier]
```

两个关键取舍：

- A tile 被两次 GEMM 共用，使每字节 shared 写入换来的 flop 从 64 提到 85.3；
- gate 与 up 复用同一块权重缓冲，而不是各给一块。两块缓冲能把 barrier 从 4 个降到 2 个，但 LDS 变成 48 KiB → 每 CU 只剩一个 block，实测 +39%。在 32 KiB 预算内，"每 barrier 的 flop" 这个指标对两种写法恰好相等（都是 1.05 MFLOP），所以单缓冲是无损的选择。

3.3 MMA shared tile 的 swizzle 布局：-9%

T.gemm 默认把 shared tile 按行主序摆放，MMA 取操作数时 16 个 lane 落在同一组 bank 上。给两块 MMA shared tile 都挂上 TileLang 自带的 swizzle 布局即可消除冲突（计算仍由 T.gemm 完成）：

```
T.annotate_layout({
    input_shared:  tilelang.maca.intrinsics.make_mma_swizzle_layout(input_shared,  vecSize=4),
    weight_shared: tilelang.maca.intrinsics.make_mma_swizzle_layout(weight_shared, vecSize=4),
})
```

五个 scope 全部启用，实测（40 次迭代）：

	Config1	Config2	Config3
无 swizzle	3.052	5.451	10.367 ms
vecSize=4	2.761	4.975	9.510 ms

vecSize 必须等于访存粒度：fp16 下 4 个元素 = 8 字节，正好是 emitter 取操作数的宽度 （maxPhase = min(16, 64/vecSize) = 16）；改成 8 反而更差（2.823 / 5.068 / 9.669），改成 2 慢 3 倍。 这一项也让另一条常见优化直接作废：把 MMA 的 LDS 读指令数减半（k_pack=2）在 swizzle 之上一分钱都不值（2.758 / 4.981 / 9.513 对 2.761 / 4.975 / 9.510），因为两者攻击的是同一项开销。

3.4 第二块权重 tile 经寄存器暂存：把 DRAM 往返藏到 GEMM 后面（-3~6%）

单权重缓冲的代价是 copy up_w[k] 夹在两个 barrier 之间，它的 DRAM 往返完全暴露在关键路径上；而它不能有自己的 shared buffer（§3.2 的 48 KiB 问题）。规则禁止异步拷贝，本 build 的 MACA 异步拷贝在超过 128 线程时还会静默算错 —— 只剩寄存器有余量。

做法：把 up 权重 tile 先读进一个 fragment（128×64 fp16 / 256 线程 = 32 个元素 = 16 个 VGPR）， 放在 barrier 之前，等 gate 的 GEMM 算完之后再写进 shared：

```
T.copy(up_w[expert_id, ...], w_stage)          # global →寄存器，在 barrier 之前发出
T.gemm(input_shared, weight_shared, gate_local, transpose_B=True)
T.copy(w_stage, up_weight_shared)              # 寄存器 →shared，关键路径上只剩一次 LDS 写
T.gemm(input_shared, up_weight_shared, up_local, transpose_B=True)
```

barrier 数不变；"写在读之后"这条不变量也不变，所以它是构造上安全的，不依赖测试运气。

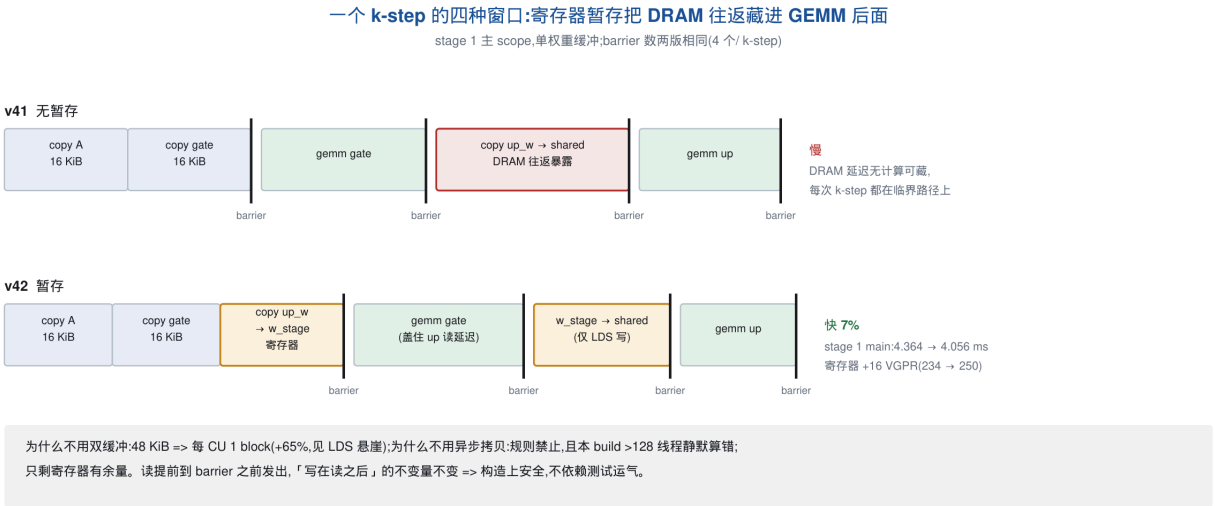


图 3-4 一个 k-step 的四种窗口:上为 v41(无暂存,DRAM 往返暴露),下为 v42(读提前进寄存器,被 gate GEMM 盖住)

mcTracer 逐 scope（Config3，单位 ms / 每线程寄存器）：

scope	swizzle 之后	+ stage 1 暂存	最终	寄存器
stage 1 main (128 行)	4.364	4.092	4.056	234 → 250

stage 1 tail (64 行)	1.893	1.733	1.744	154 → 170
stage 2 main (配对)	2.306	2.306	2.217	244 → 238
stage 2 tail	0.916	0.917	0.865	154
合计	9.480	9.049	8.882	全部 2 block/CU

3.5 K-loop 展开常数与寄存器预算的耦合

256 线程下每 CU 两个 block 的寄存器上限是 256 个/线程。stage 2 主 scope 原本就占 244 个，加上 §3.4 的 16 个就会翻过去，实测掉到每 CU 一个 block、慢 8%。解法不是放弃暂存，而是把 stage2_unroll 从 2 降到 1 —— 它本身是中性的（9.031 对 9.008），作用是腾寄存器，于是同一个 暂存反而快 3.9%（scope 2.306 → 2.217，寄存器 238）。反过来 stage 1 的展开因子越大越好一点： 2 → 9.000、4 → 8.928、8 → 8.898、16 → 8.908。

因此正式配置是 stage1_unroll = 8、stage2_unroll = 1，两条必须一起改才有效。这也是这个 kernel 上"改一个参数看一个数"不成立的地方：展开因子与寄存器暂存通过寄存器预算耦合在一起。

3.6 运行时薄块分流：一对 64 行 tile 的 scope

倾斜路由让约三分之一的 block 只有 16 -64 行有效，用 128 行 tile 处理它们要多算一倍的 padding 行。 第二对 scope 用 64 行 tile 专门接管它们，每个 block 在运行时由 group_sizes 与 group_padded_offsets 算出自己的有效行数来决定走哪一对 scope（不依赖任何关于路由分布的假设）。

代价是两次额外 launch，而收益随 block 数增长，所以门槛完全由 shape 决定：

```
tail_rows = 64 if (blocks >= 64 and blocks < 2 * experts and 4 * (blocks - experts) >= blocks) else 0
```

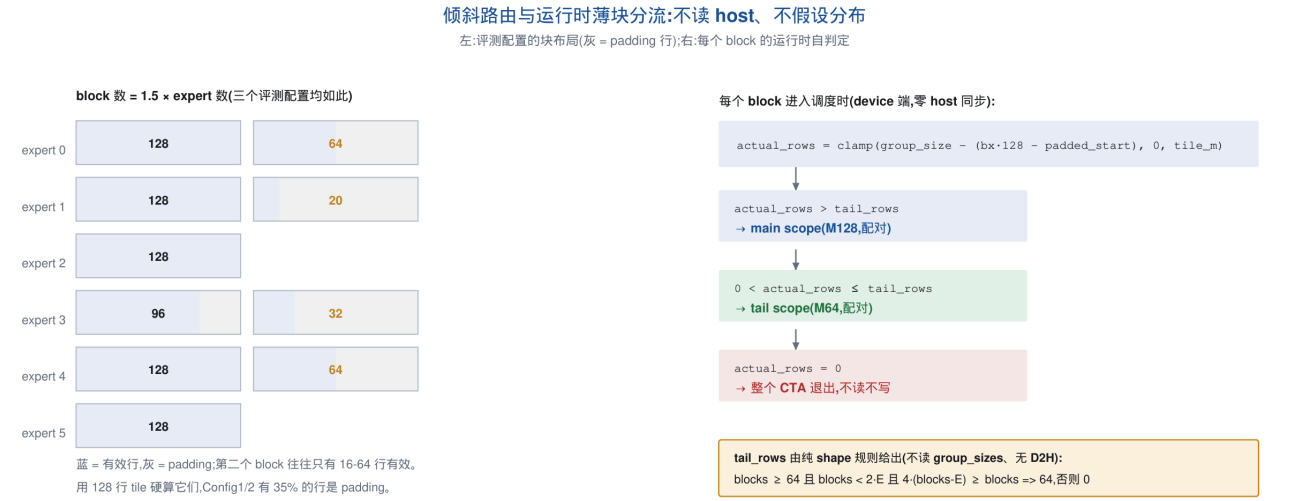


图 3-5 倾斜路由与运行时分流:左为评测配置的块布局,右为每个 block 的 device 端自判定

实测：Config3 开启比关闭快 2.2%（早期版本上曾值 7%）；Config2 开启反而 +4.7%、Config1 +19.5%（它们的 block 数太少，摊不平额外 launch），所以只有 Config3 会启用。 blocks < 2·experts 说明路由处在"每个 expert 占一到两个 block"的区间，此时 blocks - experts 正好是需要第二个（通常很薄）block 的 expert 数 —— 这条规则在 27 个测试布局上与"精确统计行数"的 旧实现给出完全相同的决定，但它只看 shape，不需要把 group_sizes 读回 host（§3.9）。

3.7 stage 2 的 n-tile 配对与 epilogue

配对：让每个 stage-2 block 负责两个相邻的 n-tile，两者共用同一块 A tile 和同一块权重缓冲， 于是每个 GEMM 灌进 shared 的字节从 (128+128)·bk·2 降到 (64+128)·bk·2 —— 正是 stage 1 已经有的 比例。两个累加器要求 512 线程时会退化，实测在 256 线程下最优：Config3 stage 2 单阶段 3.522 → 3.289（-6.6%

）、Config2 1.779 → 1.660（-6.7%）、Config1 1.030 → 0.997（-3.2%）。

epilogue 直接从 fragment 写回：配对之后 stage 2 的写回只占该阶段 4.7%（未配对时 13.6%）， 所以不再经 shared 中转 —— 这同时消掉了一个曾经在评测机上引发正确性问题的写后读风险。 所有写回都带 i < actual_rows 谓词，padding 行永不写入。Config1（hidden == 2048）额外走两个 特化：stage 1 直接融合写出 up · gate · sigmoid(gate)（不回写两个 fragment）、stage 2 把路由权重 转成 fp16 后只给有效行赋值，两项合计约 -1%。

3.8 让后端删掉内层循环里的边界保护

expert_id = T.min(T.max(group_idx_for_bx[bx], 0), num_experts - 1) 的钳位不是防御性代码：它让 后端能证明权重访问在界内，从而删掉内层 k 循环里的保护分支，实测值 3 -8%。同理路由权重的索引用 T.min(..., total_valid_tokens - 1) 夹住。生成的 MACA 源码里可以直接核对这一点。

3.9 只依赖 **shape** 的调度决策（合规且无同步）

早期版本把 (block_m, tail_rows) 的决定按 shape 记忆在字典里，并且为唯一的 Config3 shape 写了一个默认值 —— 前者形式上就是"跨调用按 shape 复用"，后者形式上就是"针对已知 testcase 硬编码"，都撞在 组委会的禁止清单上，v40 已全部删除，换成 §3.6 那条纯 shape 规则。

顺带的好处是计时区间里不再有任何 D2H 同步：原来第一次调用需要 group_sizes.cpu()。注意"删缓存"与"删 host 读取"必须一起做 —— 只删缓存而保留 host 读取会让 Config2 慢 2.4%、Config3 慢 1.0% （每次调用一次同步 D2H）。

留下的三处 shape 分支都是结构参数而非已知 testcase：hidden == 2048 的 epilogue/展开分支、 hidden == 7168 and stage2_pair == 2 的光栅化 panel、n-tile 数 ≥ 32 时的 stage-1 panel。kernel 本来 就按 shape JIT 编译，这与任何 autotuner 按维度选调度是同一件事，且都不改变数值结果。 _KERNEL_CACHE 缓存的是编译产物，_WORKSPACE_CACHE 缓存的是 scratch 显存，两者都不含计算结果。

3.10 生成代码剖析：设计落到机器上的样子

kernel.get_kernel_source() 给出的 MACA C 源码是检查每一项设计是否落地的最终物证；三种评测形状的源码长度与 md5 已记录在 02_日志与结果/02_check_equiv.log。以 Config2 的 stage 1 为例，一个 k-step 展开成：

高层语法	生成代码
T.use_swizzle(8)	const dim3 blockIdx = tl::rasterization2DRow<8>(); —— blockIdx 在 kernel 内被软件重映射
T.copy(x tile, input_shared)	每线程 8 次 uint2（8 B）全局读 + swizzle 地址（(tid&15)^(tid>>4)) 的 shared 写；权重读外包 condval 边界保护
T.copy(up_w tile, w_stage)	4 次 uint4（16 B）全局读进寄存器数组 half_t w_stage[32]，位于 barrier 之前
T.gemm(...)	A/B fragment 以 8 B LDS 从 shared 重载（每 ki 每线程 16+16 个 half），随后 64 次 warp 级 __builtin_mxc_mma_16x16x16f16（D = 每线程 float32x4）
累加器 gate_local[64] 等	T.alloc_fragment → 寄存器数组（128×128 fp32 / 256 线程）
if i < actual_rows 写回	生成真正的谓词分支，__float22half2_rn 打包成 uint2 存储
路由权重夹取	routed_expert_weights[min(idx, 4543)] —— T.min 直接变成常量 4543
Config1 倒序 bx	无运行时取反：group_idx_for_bx[(23 - blockIdx.x)]，地址常量折叠为 +6029312 - bx·262144
T.unroll(unroll_factor=8)	#pragma unroll 8（交给后端）；stage2_unroll=1 → #pragma unroll 1（显式禁止展开省寄存器）

每个 k-step 恰好 4 个 __syncthreads()、48 KB DRAM 读（A + gate + up）、约 192 KB LDS 流量、 2×64 次 MMA（每 warp 调用 2048 MAC，512 条指令精确覆盖 128×128×64 tile）—— §3.2/§3.4 的每一个设计决策都能在这一层对上号。

---

4. 正确性与精度

检查	方法	结果
数值正确性	24 个布局，与 fp32 参考实现逐元素比对（02_日志与结果/03_func_test.log）	全部 PASS
padding 未写	输出缓冲预填哨兵，断言每个 padding 元素仍是哨兵	全部 clean
精度余量	三个评测 shape 最差绝对误差 2.6e-3，容差 atol=rtol=0.05	约 19 倍余量
任意路由	每个 scope 在运行时读 group_sizes / group_padded_offsets 算自己的行数	与分布无关
官方 harness	03_源码与复现/compat/fusedmoe_benchmark.py 的两个功能配置（02_日志与结果/06）	[OK] [OK]

三点值得强调：

- 不做量化。 我们完整实现并实测过 int8 量化路线（只量化 A 操作数、per-16 scale）：按评测的真实判据 |d| ≤ 0.05 + 0.05·|ref|，最好情况余量只有 1.1 –1.45 倍，而 现在是 ~19 倍；而且实测带 dequant 的 stage 1 反而慢 66%。结论是不做（§6.2）。
- padding 行的处理是踩过坑的：一个没加谓词的 T.copy epilogue 曾在本地"只查有效行"的检查里全部通过，却在评测机上挂在 (2358, 1727) —— 参考值就是那块内存的旧内容。因此本地检查加了 哨兵断言，正式实现的每一处写回都有谓词。
- up_logits 的 padding 行内容是未定义的（stage 1 不写它们），stage 2 读到这些行只会污染自己 accumulator 里同样被谓词挡住的行，不影响任何有效输出。

---

5. 性能结果

5.1 三个评测配置

本机（MetaX C500 的 25% sGPU 切片）40
次迭代、单进程、板级空闲时测得（02_日志与结果/04_perf_test.log）：

	本地耗时	有效吞吐*	线上得分
Config1（16 experts, 2048/8192）	2.678 ms	85.4 TFLOP/s	81.7
Config2（32 experts, 7168/2048）	4.730 ms	84.6 TFLOP/s	80.7
Config3（64 experts, 7168/2048）	8.907 ms	89.9 TFLOP/s	80.7

* 按有效 token 计的 flop / 时间；若把 padding 行也算进去（kernel 实际执行的 MMA），stage 1 主 scope 的吞吐为 118 TF/s。题目基线为 50 分 / 66 ms。



图 5-1 性能结果:三个评测配置(左)与 starter kit 模板的同环境对照(右)

5.2 逐 kernel scope 分解（mcTracer，02_日志与结果/05_kernel_profile.log）

config	scope	ms / 次调用	寄存器/线程	shared
Config1	stage 1（128 行）	1.705	250	32 KiB
Config1	stage 2（配对）	0.963	236	32 KiB
Config2	stage 1	3.140	250	32 KiB
Config2	stage 2	1.579	238	32 KiB
Config3	stage 1 main / tail	4.055 / 1.750	250 / 170	32 / 24 KiB
Config3	stage 2 main / tail	2.234 / 0.867	238 / 154	32 / 24 KiB

每个 scope 都是每 CU 两个 block 驻留（寄存器 ≤ 256、LDS ≤ 32 KiB），这是要保护的不变量； stage 1 占总时间的 64 -65%。

5.3 与 starter kit 模板实现的同环境对照（02_日志与结果/07、02_日志与结果/08）

- 显存条件说明：本次实验使用 MetaX C500 的 16 GiB sGPU 切片（实测可用显存约 15.22 GiB），
- 未能使用题目环境中的 64 GiB 卡。starter kit 原始 full harness 在 bs=4 / bs=8 配置下会因自身
- repeat / scatter_reduce 临时张量额外申请约 7 GiB 而 OOM；这不是提交算子功能失败。下表采用
- compat/perf_slice.py 的 bs=1 / bs=2 可运行对照，模型形状和实现保持一致。

用组委会自己的 harness、同一台机器、同一组配置，只切换实现（MOE_IMPL=template）：

配置	模板 kernel	本方案	加速
7168 / 2048 / 8 experts / topk 4 / 1×8192	117.29 ms	48.38 ms	2.42×
3584 / 1024 / 4 experts / topk 2 / 2×4096	18.40 ms	9.62 ms	1.91×

该口径把 harness 每次迭代的权重 clone（8-expert 配置约 700 MB）、Python 侧路由排序与 scatter_reduce 都算进去了，kernel 本身的差距比 2.42× 更大。

原始 bs=4 / bs=8 full harness 的失败记录保留在 02_日志与结果/06_official_functional.log 和 08_official_baseline.log，错误发生在 harness 的临时张量分配阶段；算子自己的 27 布局功能测试和 三个 graded shape 检查均通过，详见 §4 和 03_func_test.log。

5.4 优化历程（本地 oj*exact 三项，ms）

版本	关键改动	Config1	Config2	Config3

v22	128×128×64 分组 GEMM + stage 2 配对 + tail 分流	2.80	4.99	9.14
v36 -v38	纯 TileLang 化、fast math、Config1 epilogue 融合与 K-loop 展开	3.05	5.46	10.39
v40	去掉跨调用记忆与 testcase 默认值，改为纯 shape 规则（设备代码不变）	3.05	5.45	10.39
v41	MMA shared tile 的 swizzle 布局（§3.3）	2.761	4.975	9.510
v42	权重 tile 寄存器暂存 + 展开因子重配（§3.4/§3.5）	2.678	4.734	8.905

v36 -v38 一行的绝对值变大，是因为那一轮按赛题要求删掉了两条不合规的旁路（动态改写 GEMM lowering、 用 PyTorch @ 承担一部分数学），把它们换成纯 TileLang 语法后先付了一次成本，随后由 v41/v42 补回。

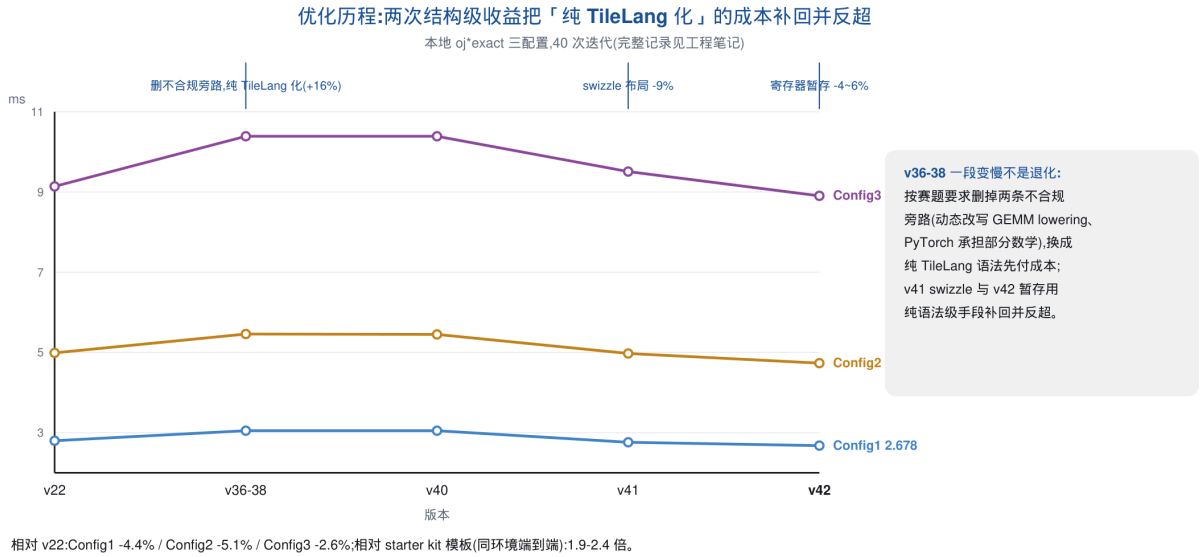


图 5-2 优化历程:v41 的 swizzle 与 v42 的寄存器暂存把"纯 TileLang 化"的成本补回并反超

6. 优化方法论与实测数据

这一节是本方案最想传达的部分：在这个 kernel 上，哪些判断方法是可靠的。完整记录（含每一项的 数字）在 01_技术方案/附录_工程笔记.md，此处只摘方法与结论。

6.1 唯一站得住的成本模型：block-k-step 计数

对固定的 tile 形状，时间 = (活跃 block 数 × k-step 数) × 该 tile 的每步常数，在三个配置、 七个 scope 上误差 ≤ 1%：

tile	每 (block, k-step) 摊薄成本
128×128×64（两个主 scope）	36.5 ns
64×128×64（两个 tail scope）	30.4 ns
128×64×64	22.7 ns
192×64×64	36.2 ns

例：Config1 = 24×64×32 + 24×8×128 = 73728 步 × 36.5 ns = 2.69 ms（实测 2.678）； Config3 四个 scope 分开算得 8.89 ms（实测 8.905）。任何改动先数它改变了多少 block-k-step。

但那个常数推不出来。 我们先后拟合过三个线性模型（α+ βbytes + γflop、纯 bytes、 LDS 读+写），每加入一个没参与拟合的 tile 形状就崩一次：(128,64) 比字节模型快 18%、(192,64) 慢 60%，把 MMA 的 LDS 读砍掉 18%（让 gate/up 共用一套 A fragment）反而慢 6.5%。新形状必须实测。

6.2 动手之前必做的三个测量

- 看寄存器余量（mcTracer）。两个主 scope 是 250 与 236–238，而每 CU 两 block 的上限是 256，任何需要多暂存一块 tile（16 VGPR）的想法在写代码之前就已经死了 —— 这一条一次性解释了 7 个 失败变体。生成源码里是 __launch_bounds__(256, 1)，编译器并没有被要求留在 256 以内。
- 数每个 scope 的活跃 block 数。活跃 block 少的 launch 吃不满带宽：768 个活跃 block 时达到 4.82 TB/s 的 LDS 流量，512 个 4.54，256 个只有 3.60。任何"把工作按类别拆到新 scope"的方案都要 先估两边各剩多少活跃 block（这正是 tail 分流只在 Config3 划算的原因）。
- 把线上分数反解成时间，确认"本地 1% 值多少分"。见 §6.3。

6.3 本地与评测机的标定

用线上给出的三个分数与曲线 ratio ≈ 0.5385 + 0.1331·log2(speedup) 反解，得到评测机上的用时约为 2.57 / 4.55 / 8.86 ms，而本地是 2.678 / 4.730 / 8.907 —— 评测机一致地只快 0.5–4%，尽管它有 104 个 CU 而本地切片只有 26 个。两个推论：

- 两台机器的瓶颈是同一个（同一条 HBM/L2 通路），多出来的 CU 对这个访存受限的算子几乎无用；因此本地测量可以直接迁移，也因此我们没有为"评测机更大"做任何特化 —— 历史上两次这样的赌注（256 行 tile、block_k=128）都以本地测量的方向被否决。
- 早期版本里"线上时间比本地多 0.7–1.2 ms"的 harness 开销现在已不存在，本地省下的百分比可以直接换算成分数（约 5.3% 时间 = 1 分）。

6.4 试过并被否掉的方向（全部带实测数字）

方向	实测	原因
更大的 M tile（256 行）	+22%	48 KiB LDS → 每 CU 一个 block
block_k=128 / block_k=96	+27% / +66%	同上；L2 足迹翻倍
block_k=32 的一切变体	+38~70%	64 字节的 DRAM 行段效率只有 128 字节的 76%
两块权重缓冲	+39%	48 KiB LDS；而"每 barrier 的 flop"并没有变好
512 / 1024 线程	+34% / +54%	wave tile 退化成 32×64 / 32×32
把 128 行与 64 行 tile 合进一个 scope	+53%	编译器为两个互斥分支都分配累加器
gate/up 共用一套 A fragment（手工驱动 emitter）	+6.5%	LDS 读少 18%，但跨 barrier 多占 32 VGPR，翻过 256
A / gate tile 也过寄存器暂存	+4~9%	同上，寄存器已满
跨 k-step 的寄存器软流水	+2~5%	同上
把一个 expert 的两个 block 合成 192 行 tile	+1.7~26%	相邻 block 的权重读本来就命中 L2，省不出流量，而 bn 减半让 LDS 流量/flop 涨 26%
int8 量化 A 操作数	精度余量掉到 1.1–1.45 倍	见 §4；且带 dequant 的 stage 1 慢 66%
权重预处理（relayout）	净亏	读一遍写一遍就是 2W 流量，而整个 kernel 只搬约 W
PyTorch 逐 expert / bmm	慢 1.6–2.0 倍	expert 只有 64–220 行，单 GEMM 填不满机器
MACA 异步拷贝	>128 线程静默算错	且规则禁止
T.gemm(k_pack=2)	生成源码逐字节相同	前端参数在 MACA 路径上被丢弃
光栅化 panel 2/4/16/24/32/56、FullRow/FullCol policy、交换网格轴	+0.4~+75%	现有的 panel 8（stage 1）/ 28 （stage 2 配对）最优

TL_DISABLE_SAFE_MEMORY_ACCESS、warp specialization、shared memory 激进合并	±0.5%	噪声内，且第一项会去掉越界保护

最后一轮（2026-09-02）沿 25 个方向、35 个变体复测，全部为中性或负收益，因此 v42 即为定版。

7. 合规性自查

7.1 对照组委会的禁止清单

#	禁止项	自查结论
1	绕过 TileLang 直接执行设备 kernel	无。全文件只有一个 @tilelang.jit 的 T.prim_func，2 -4 个 T.Kernel scope
2	用 PyTorch 完成核心计算（matmul/mm/bmm/einsum/F.linear）	无。唯一的 torch.* 调用是 torch.empty（分配 up_logits workspace，题面明确其为可复用 workspace 且不作为评测输出）；两段矩阵乘全部是 T.gemm，SwiGLU 是 T.exp2 + 标量表达式，路由权重乘法与写回都在 kernel 内
3	用 H2D/D2H 异步拷贝规避评测	无。没有 stream / event / non_blocking；生成的 MACA 源码里 grep 不到 async / mbarrier / buffer_load_lds
4	跨调用缓存、复用或回放结果	无。v40 已删除按 shape 记忆调参结果的字典；_KERNEL_CACHE 缓存编译产物、_WORKSPACE_CACHE 缓存 scratch 显存，都不含计算结果，每次调用都按当前输入重算
5	针对已知 testcase 硬编码投机	无。v40 已删除按已知 Config3 shape 写死的默认值；留下的 shape 分支都是结构参数（见 §3.9），correctness 与 benchmark 阶段是同一条代码路径，全文件没有任何调用计数
6	T.import_source	无
7	T.call_extern 调用外部设备计算	无
8	mcTlass 等外部设备库承担计算主体	无。grep 不到 mcTlass / asm / 任何外部库

7.2 对照提交沙箱的静态规则

python 03_源码与复现/test/check_submission.py 03_源码与复现/kernel/submission.py → OK（02_日志与结果/01_check_submission.log）。 它复现了沙箱的三条规则：模块顶层只允许 import / def / class / 右侧能通过 ast.literal_eval 的赋值；只允许 import torch 与 tilelang（任何嵌套深度、包括 try/except 里也不行）；run_kernel 的参数顺序固定且不能加类型标注。文件里没有任何 monkey-patch ——make_mma_swizzle_layout 与 T.gemm 都是通过属性遍历取到的公开 TileLang 语法（组委会已明确该写法允许）。

7.3 关于 03_源码与复现/kernel/moe.py 的一处说明

moe.py 是带注释的同源实现，比 submission.py 多一个编译期开关 unpadded_rows：默认值下生成的 设备代码与 submission.py 逐字节相同；置 1 时只把三个行主序张量的行基址从 "padding 后的位置"换成 "紧凑排布的位置"，用于接入 starter kit 那套紧凑布局的 harness（03_源码与复现/compat/custom_fusedmoe.py）。提交给评测系统的 submission.py 里没有这个开关。

8. 复现方式

见 README.md §4。最小验证序列：

```
python 03_源码与复现/test/check_submission.py 03_源码与复现/kernel/submission.py       # OK
python 03_源码与复现/test/check_equiv.py 03_源码与复现/kernel/moe.py 03_源码与复现/kernel/submission.py  # EQUIVALENT
python 03_源码与复现/test/local_bench.py --module $PWD/03_源码与复现/kernel/submission.py --func-only # 全部 PASS
python 03_源码与复现/test/local_bench.py --module $PWD/03_源码与复现/kernel/submission.py \
       --only oj1exact,oj2exact,oj3exact --iters 40 --no-torch-baseline   # 性能
cd compat && python fusedmoe_benchmark.py                                  # 官方 harness [OK]
```

bash run.sh 会把 02_日志与结果/ 下的九个日志全部重新生成；bash run.sh quick 只生成关键验收日志。

---

9. 结论

结论：在"只用 TileLang 高层语法、不量化、不缓存结果"的约束下，本方案把三个评测配置分别做到 2.678 / 4.730 / 8.907 ms（线上 81.7 / 80.7 / 80.7 分，题目基线 50 分），相对 starter kit 模板实现 在同环境下端到端快 1.9–2.4 倍。核心是把 32 KiB 的 LDS 预算与 256 个寄存器的上限当作两条硬约束， 推导出唯一可行的 tile，然后用两项纯语法级的手段（MMA shared tile 的 swizzle 布局、权重 tile 的 寄存器暂存）把剩余的 bank 冲突与暴露的 DRAM 延迟各消掉一层。